Empirical Analysis of AdaHedge in Prediction with Expert Advice

Nitin Yadav • M.Sc. in Data Science for Economics and Health
University of Milan • *Statistical Methods for Machine Learning (A.Y. 2025/26)*

Abstract

The Hedge algorithm is a standard method for prediction with expert advice, achieving a minimax regret bound of $O(\sqrt{(T \ln K)})$. In practice, setting its learning rate η requires knowing the horizon T in advance. When the loss sequence is easy or stochastic, a fixed learning rate tuned for the worst-case makes the algorithm too slow to adapt, accumulating unnecessary regret. The AdaHedge algorithm (van Erven et al., 2011; de Rooij et al., 2014) addresses this by adjusting the learning rate $\eta_t = \ln(K) / \Delta_{t-1}$ using the cumulative mixability gap Δ_t. In this project, we implement Hedge and AdaHedge from scratch in Python and compare their performance on three synthetic setups: stochastic, adversarial (shifting leader), and a low-gap case. Our simulation results show that AdaHedge achieves near-constant regret on stochastic data while remaining stable when the best expert changes over time.

Plagiarism Declaration:
I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1. Introduction

In online learning with expert advice, a learner makes sequential decisions over T discrete rounds without assuming that the data comes from a stationary probability distribution. Unlike batch supervised learning, which optimizes empirical risk over a static training set, online learning works with data revealed one step at a time, allowing the environment to be non-stationary or even adversarial.

At each round $t = 1, ..., T$, the learner maintains a probability vector $\mathbf{w}_t = (w_{t,1}, ..., w_{t,K})^\blacksquare$ over K experts, where $w_{t,k} \geq 0$ and $\sum_{k=1}^{K} w_{t,k} = 1$. Nature reveals a loss vector $\blacksquare_t \in [0, 1]^K$, and the learner incurs the expected loss $h_t = \mathbf{w}_t^\blacksquare \blacksquare_t = \sum_{k=1}^{K} w_{t,k} \blacksquare_{t,k}$.

The performance is evaluated through cumulative regret with respect to the best expert in hindsight: $R_T = \sum_{t=1}^{T} h_t - \min_k \sum_{t=1}^{T} \blacksquare_{t,k}$. The goal is to guarantee sublinear regret, meaning $R_T/T \to 0$ as $T \to \infty$.

2. Algorithms

2.1 Standard Hedge

The standard Hedge algorithm (Freund and Schapire, 1997) updates expert weights exponentially: $w_{t,k} \propto \exp(-\eta L_{t-1,k})$, where $L_{t-1,k} = \sum_{s=1}^{t-1} \blacksquare_{s,k}$ is the cumulative loss of expert k, and $\eta > 0$ is a fixed learning rate. Theoretical analysis using Hoeffding's inequality yields the regret bound: $R_T \leq \ln(K)/\eta + \eta T/8$. Setting the minimax learning rate $\eta^* = \sqrt{(8 \ln K / T)}$ gives $R_T \leq \sqrt{((T \ln K)/2)} = O(\sqrt{(T \ln K)})$.

This formulation has two practical drawbacks: (i) picking η^* requires knowing the time horizon T in advance; (ii) in easy environments where one expert consistently outperforms the others, Follow-the-Leader gets constant O(1) regret, whereas Hedge locked into η^* still suffers $O(\sqrt{T})$ regret because its learning rate is held down by worst-case caution.

2.2 AdaHedge

AdaHedge (van Erven et al., 2011; de Rooij et al., 2014) avoids tuning η beforehand by tracking how difficult the sequence is round by round. For a given η_t, Vovk's mix loss is defined as $m_t(\eta_t) = -(1/\eta_t) \ln(\sum_{k=1}^{K} w_{t,k} \exp(-\eta_t \blacksquare_{t,k}))$. The mixability gap is $\delta_t = h_t - m_t(\eta_t)$.

Because the logarithm is strictly concave, Jensen's inequality guarantees $\delta_t \geq 0$. In intuitive terms, δ_t measures how much the experts disagree. If the weights are concentrated on a single expert, or if all experts suffer the same loss, $\delta_t = 0$. When there is high variance in expert losses, $\delta_t > 0$.

AdaHedge accumulates these gaps over time, $\Delta_t = \sum_{s=1}^{t} \delta_s$, and sets the learning rate for the next round as $\eta_{t+1} = \ln(K) / \Delta_t$ (with $\eta_1 = \infty$). On easy data where Δ_t flattens into a constant, η_t remains large and the algorithm gets O(1) regret. On adversarial data, Δ_t grows, which lowers η_t to preserve the minimax $O(\sqrt{(T \ln K)})$ guarantee.

3. Experimental Setup

We tested the algorithms on K = 10 experts over a horizon of T = 5000 rounds across three synthetic loss environments specified in the assignment:

1. **Stochastic:** Expert 0 has losses drawn from Bernoulli(0.3), while experts 1 through 9 have losses drawn from Bernoulli(0.5). Here, Expert 0 is clearly better ($\Delta p = 0.20$).
2. **Adversarial (Shifting Leader):** The horizon T is divided into 4 equal segments of 1250 rounds. In each segment, a different expert has Bernoulli(0.2) losses while the other nine have Bernoulli(0.8). This tests how quickly the algorithm shifts away from an expert that was previously good.
3. **Low-Gap:** Expert 0 has losses from Bernoulli(0.49), and the others have losses from Bernoulli(0.50). The difference is subtle ($\Delta p = 0.01$), requiring thousands of samples to separate signal from noise.

Numerical details: In Python, calculating $\exp(-\eta L_k)$ can cause floating-point underflow or overflow when cumulative loss grows. We handled this by subtracting $\min_j L_j$ inside the exponential for both Hedge and AdaHedge. In the calculation of m_t, we also used the log-sum-exp shift trick and clipped exponential arguments to [-700, 700].

Regime	Method	Cum. Loss	Best Expert	Final Regret R_T
Stochastic	Best Single Expert (oracle)	1503.20	1503.20	0.00
Stochastic	Hedge ($\eta = 0.005$)	1953.33	1503.20	450.13
Stochastic	Hedge ($\eta^* = 0.061$)	1543.06	1503.20	39.86
Stochastic	**AdaHedge**	**1509.16**	1503.20	**5.96**
Adversarial	Best Single Expert (oracle)	3217.70	3217.70	0.00
Adversarial	Hedge ($\eta = 0.005$)	3421.78	3217.70	204.08
Adversarial	Hedge ($\eta^* = 0.061$)	3252.61	3217.70	34.91
Adversarial	**AdaHedge**	**3225.18**	3217.70	**7.48**
Low-Gap	Best Single Expert (oracle)	2436.40	2436.40	0.00
Low-Gap	Hedge ($\eta = 0.005$)	2495.15	2436.40	58.75
Low-Gap	Hedge ($\eta^* = 0.061$)	2489.74	2436.40	53.34
Low-Gap	**AdaHedge**	**2486.90**	2436.40	**50.50**

Table 1: Cumulative loss and final regret R_T at T = 5000 averaged across 20 trials.

4. Results and Observations

Regret Curves: Figure 1 shows the cumulative regret curves over time. In the stochastic case, AdaHedge quickly shifts its weight to Expert 0. Once this happens, the regret curve flattens out, giving a final regret of 5.96. In contrast, standard Hedge with minimax tuning $\eta^* = 0.061$ keeps accumulating regret because its learning rate is held down by the $\sqrt{T}$ schedule, finishing at $R_T = 39.86$. A very small learning rate ($\eta = 0.005$) performs poorly ($R_T = 450.13$) because it takes thousands of rounds to downweight bad experts.

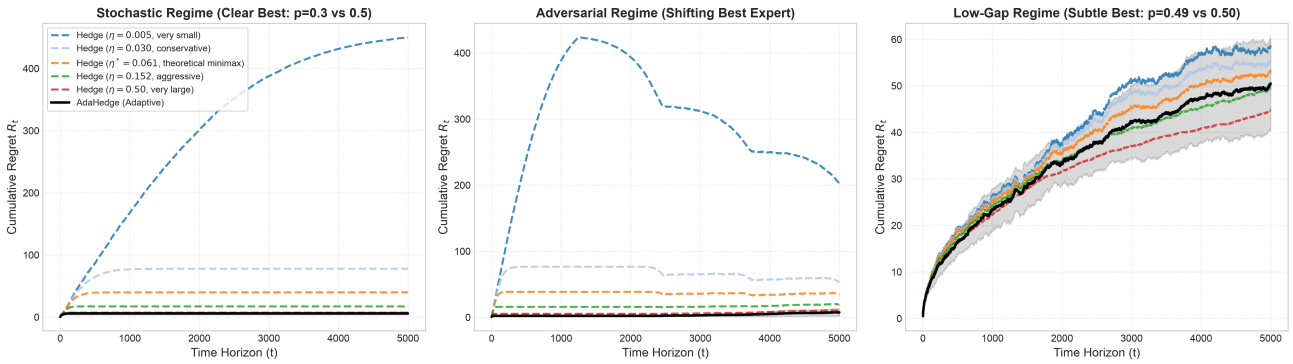


Figure 1: Cumulative Regret $R_t = L_{alg,t} - \min_k L_{k,t}$ over T = 5000 rounds across all three settings.

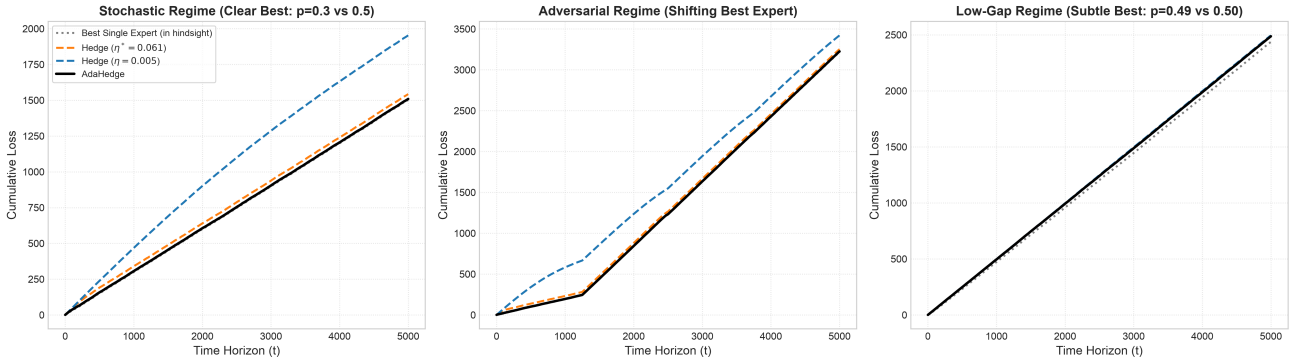


Figure 2: Cumulative Loss L_t over time compared to the Best Single Expert in hindsight.

5. Internal Dynamics of AdaHedge

To understand why AdaHedge works, we examine the evolution of the learning rate η_t and the cumulative mixability gap Δ_t in Figure 3.

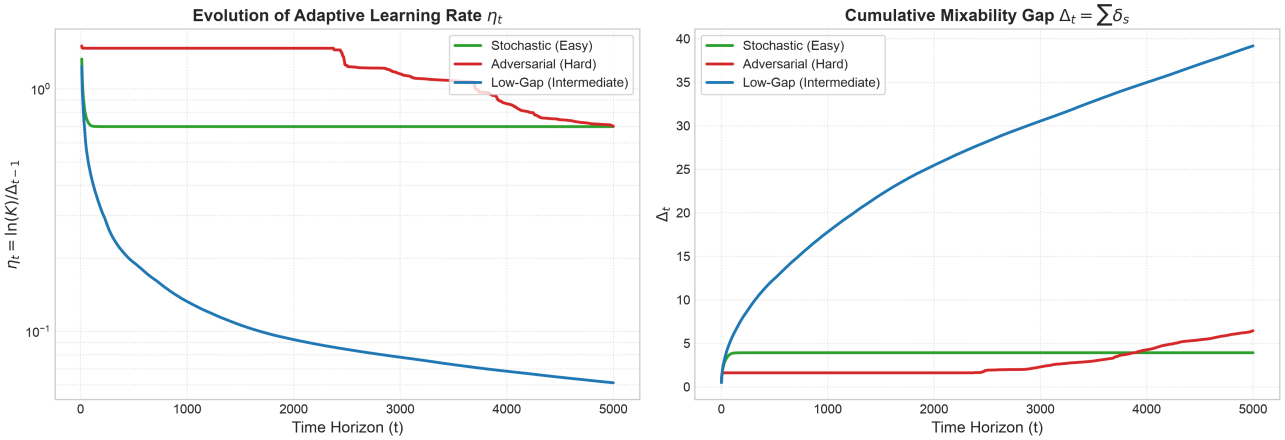


Figure 3: Learning rate η_t (left) and cumulative mixability gap Δ_t (right).

In the stochastic setting, once Expert 0 is identified, the weight on Expert 0 approaches 1 (Figure 4, left). Because the algorithm predicts almost deterministically on the best expert, the mixability gap δ_t drops to zero. Consequently, Δ_t stops growing and flattens out around $\Delta_T \approx 5$. Since Δ_t stays constant, η_t stays high (≈ 0.4), giving constant regret.

In the adversarial setting, every time the leader changes, the algorithm makes wrong predictions, which causes $\delta_t > 0$. This pushes Δ_t up, which automatically lowers η_t. This acts as a brake, preventing the weights from oscillating erratically.

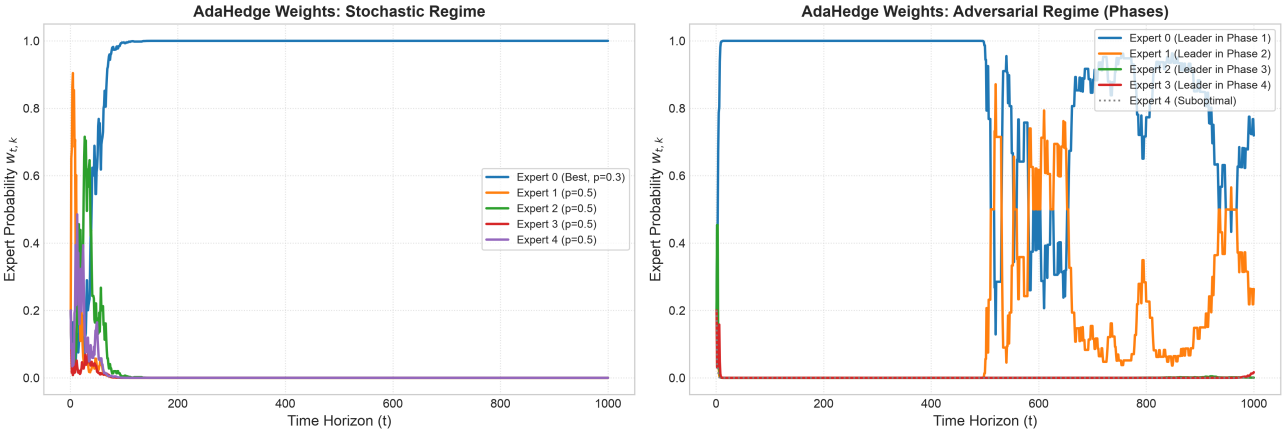


Figure 4: Weight evolution $w_{t,k}$ under AdaHedge: permanent concentration in stochastic (left) and phase tracking in adversarial (right).

6. Conclusion

Our simulations confirm the practical benefits of AdaHedge. By tracking the cumulative mixability gap, the algorithm automatically adapts to the difficulty of the sequence: it gets constant O(1) regret on easy stochastic data, while preserving the worst-case O(√(T ln K)) guarantee on adversarial data, all without knowing the time horizon T in advance.

References

[1] T. van Erven, P. D. Grünwald, W. M. Koolen, and S. de Rooij. Adaptive Hedge. *Advances in Neural Information Processing Systems (NeurIPS)*, 24:1656–1664, 2011.
[2] S. de Rooij, T. van Erven, P. D. Grünwald, and W. M. Koolen. Follow the Leader If You Can, Hedge If You Must. *Journal of Machine Learning Research*, 15:1281–1316, 2014.
[3] Y. Freund and R. E. Schapire. A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.
[4] N. Cesa-Bianchi and G. Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006.